



PROMPT ENGINEERING AND ARTIFICIAL INTELLIGENCE · 2º SEMESTRE

RAG avançado — chunking estratégico, reranking e RAGAS

Aula 07/14 — Módulo 2: RAG · 🏁 Entrega CKP02

👤 Prof. Jorge Luiz Gomes

🕒 1h40min

📊 faithfulness · answer_relevancy

🏁 CKP02 entrega

Agenda da Aula

- 1 Por que chunk_size importa — o trade-off precisão vs. contexto 15 min
- 2 Estratégias de chunking — Recursive (padrão), Semantic, Parent-Document Retriever 15 min
- 3 Reranking com cross-encoder — reordenar os chunks recuperados por relevância real 10 min
- 4 RAGAS — faithfulness, answer_relevancy, context_precision. LLM-as-judge 15 min
- 5 🚀 Demo — mesmo RAG com chunk_size 256, 512 e 1024 · comparar métricas RAGAS 15 min
- 6 🖥️ Lab — otimizar RAG do grupo com 2 estratégias + calcular RAGAS (55% lacunas) 20 min
- 7 🏁 CKP02 — entrega + alinhamento final com o grupo 10 min

🎯 OBJETIVO DA AULA

Sair da fase "funciona" para a fase "funciona bem". Medir objetivamente a qualidade do RAG com RAGAS, experimentar duas estratégias de chunking e documentar qual configuração entrega melhores resultados para o domínio do grupo. Isso é o CKP02.

Glossário

SemanticChunker

Estratégia de chunking que divide o texto nos pontos onde a similaridade semântica entre sentenças consecutivas cai abaixo de um limiar — em vez de cortar a cada N caracteres fixos.

Cross-encoder

Modelo que recebe pergunta e chunk juntos e gera um único score de relevância — mais preciso, porém mais lento, que a similaridade vetorial do retriever inicial.

Analogia: comparar dois textos lado a lado, em vez de apenas medir a distância entre pontos num mapa.

RAGAS

Framework de avaliação de sistemas RAG que usa LLM-as-judge para medir faithfulness e answer_relevancy sem precisar de respostas de referência anotadas manualmente.

answer_relevancy

Métrica RAGAS que mede se a resposta realmente aborda o que foi perguntado — mesmo uma resposta factualmente correta pode ser irrelevante para a pergunta.

Parent-Document Retriever

Busca chunks pequenos (alta precisão na similaridade) mas retorna ao LLM o documento "pai" maior que os contém — combina precisão de busca com contexto completo.

Reranking

Segunda passada de filtragem: o retriever busca um conjunto amplo (ex: $k=10$) e o cross-encoder reordena por relevância, mantendo só os melhores (ex: $\text{top}_n=3$).

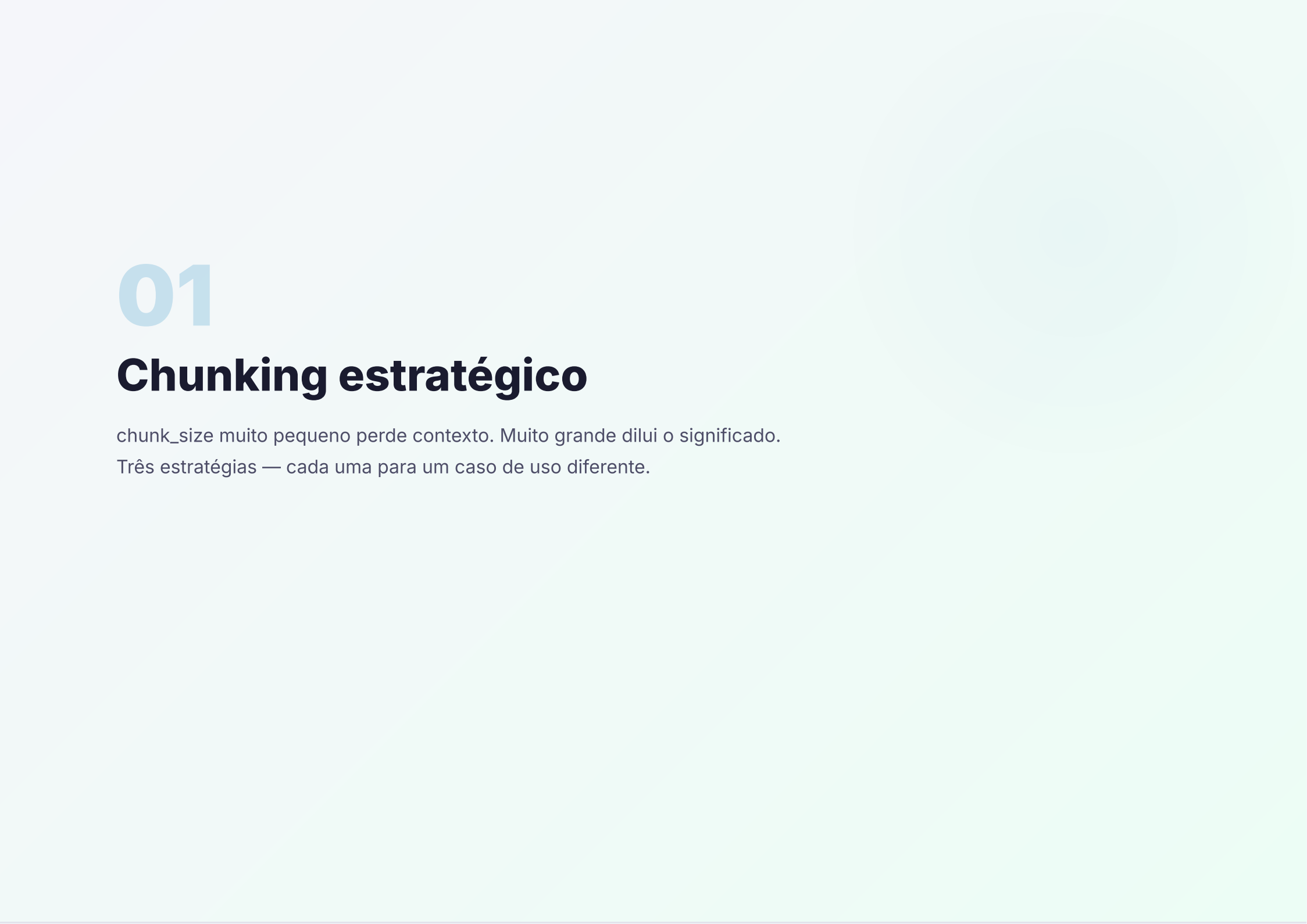
faithfulness

Métrica RAGAS que verifica se cada afirmação da resposta pode ser encontrada no contexto recuperado — detecta alucinação do modelo.

LLM-as-judge

Uso de um modelo de linguagem para avaliar a qualidade de uma resposta gerada por outra IA, substituindo (ou complementando) a avaliação humana manual.

Analogia: um segundo modelo atua como corretor de prova do primeiro.



01

Chunking estratégico

chunk_size muito pequeno perde contexto. Muito grande dilui o significado.

Três estratégias — cada uma para um caso de uso diferente.

O trade-off do chunk_size — precisão vs. contexto

O tamanho do chunk controla o quanto de texto cada embedding representa. Pequeno demais perde contexto; grande demais dilui o significado e prejudica a busca:



chunk_size pequeno (200–300 chars)

Embedding de alta precisão para aquele trecho — mas cada chunk tem pouco contexto. Uma pergunta que depende de duas frases em sequência pode recuperar só a primeira. Bom para: FAQs, bullets, definições curtas.



chunk_size médio (600–1000 chars) ← sweet spot padrão

Equilíbrio entre precisão e contexto para a maioria dos domínios. 1–2 parágrafos. O que foi usado na Aula 06 (800 chars). Funciona bem para manuais, artigos e regulamentos.



chunk_size grande (1500–2000 chars)

Contexto rico — bom para documentos com parágrafos interdependentes. O embedding "médio" de um trecho longo tende a ser menos específico, prejudicando a busca. Bom para: narrativas longas, laudos completos.



REGRA PRÁTICA — DEPENDE DO DOCUMENTO

Não existe chunk_size universal. Para o CKP02: teste 256, 512 e 1024 com o seu corpus, meça o RAGAS de cada configuração e escolha a melhor. A demo desta aula mostra exatamente esse processo.

Três estratégias de chunking — quando usar cada uma

ESTRATÉGIA 1

Recursive Character

Divide por parágrafos, linhas, sentenças e caracteres — nessa ordem de preferência.

Padrão do semestre.

✅ **Uso:** qualquer documento

ESTRATÉGIA 2

Semantic Chunker

Agrupa frases por similaridade de embedding — quebra onde o significado muda, não onde tem "\n\n". Chunks mais coerentes semanticamente.

✅ **Uso:** artigos científicos, laudos

ESTRATÉGIA 3

Parent-Document Retriever

Indexa chunks pequenos (200 chars) para busca precisa, mas retorna o chunk pai (1000 chars) para o LLM — melhor dos dois mundos.

✅ **Uso:** documentos densos

SEMANTIC_CHUNKER.PY

```
from langchain_experimental.text_splitter import SemanticChunker

# SemanticChunker usa o modelo de embedding para decidir onde quebrar
semantic_splitter = SemanticChunker(
    embeddings, # mesmo OllamaEmbeddings da Aula 06
    breakpoint_threshold_type="percentile", # quebra nos 95% de maior divergência
)
chunks_sem = semantic_splitter.split_documents(paginas)
print(f"Chunks semânticos: {len(chunks_sem)}")
print(f"Tamanho médio: {sum(len(c.page_content) for c in chunks_sem)/len(chunks_sem):.0f} chars")
```

Parent-Document Retriever — o melhor dos dois mundos

Busca com chunks pequenos (precisão alta), retorna o chunk pai grande (contexto rico) para o LLM:

PARENT_DOCUMENT_RETRIEVER.PY

```
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore

# Splitter pequeno – para indexação e busca (precisão)
child_splitter = RecursiveCharacterTextSplitter(chunk_size=200)

# Splitter grande – para o contexto enviado ao LLM (riqueza)
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=1000)

# Armazenamento para os chunks pai (não vai no ChromaDB)
store = InMemoryStore()

retriever_pd = ParentDocumentRetriever(
    vectorstore=db,          # ChromaDB com chunks de 200 chars
    docstore=store,         # store com chunks de 1000 chars
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)
retriever_pd.add_documents(paginas)

# Como funciona:
# 1. Query → ChromaDB busca nos chunks de 200 chars (alta precisão)
# 2. Encontra chunk filho → recupera o chunk PAI de 1000 chars do store
# 3. Retorna o chunk de 1000 chars para o LLM (contexto rico)
docs = retriever_pd.invoke("prazo de garantia")
print(f"Docs retornados: {len(docs)}")
print(f"Tamanho do 1º: {len(docs[0].page_content)} chars") # → ~1000
```

02

Reranking com cross-encoder

O retriever recupera candidatos por similaridade cosseno. O cross-encoder avalia cada par (query, chunk) individualmente — mais lento, mas mais preciso.

Reranking — o segundo filtro de relevância

O ChromaDB faz busca por similaridade vetorial (rápida, bi-encoder). O cross-encoder avalia cada par individualmente — mais caro, mas captura nuances que o embedding não captura:

SEM RERANKING (RETRIEVER DIRETO)

1. [0.91] "prazo de entrega: 30 dias"
2. [0.87] "entrega e devolução: prazos"
3. [0.84] "cláusula de entrega expressa"
4. [0.82] "política de entrega regional"
5. [0.81] "processo de entrega ao cliente"

→ Top-3 enviado ao LLM

COM RERANKING (CROSS-ENCODER)

1. [0.98] "prazo de entrega: 30 dias"
2. [0.95] "cláusula de entrega expressa"
3. [0.71] "entrega e devolução: prazos"
4. [0.43] "política de entrega regional"
5. [0.31] "processo de entrega ao cliente"

→ Reordenado: chunks 1,2,3 novo → LLM

RERANKING.PY

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain_community.cross_encoders import HuggingFaceCrossEncoder
from langchain.retrievers.document_compressors import CrossEncoderReranker

# cross-encoder leve, gratuito (roda em CPU no Colab)
cross_encoder = HuggingFaceCrossEncoder(model_name="cross-encoder/ms-marco-MiniLM-L-6-v2")
compressor = CrossEncoderReranker(model=cross_encoder, top_n=3)

# Combina retriever (busca ampla, k=10) + reranker (filtra para top_n=3)
retriever_rerank = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=db.as_retriever(search_kwargs={"k":10}), # busca ampla
)

# Trocar o retriever na chain RAG é tudo que precisa mudar
```

03

RAGAS — medir a qualidade do RAG

faithfulness · answer_relevancy · context_precision. LLM-as-judge: o próprio modelo avalia a qualidade das respostas geradas.

RAGAS — Reference-free Automated Evaluation of RAG

RAGAS é um framework de avaliação que usa **LLM-as-judge**: o próprio modelo avalia a qualidade das respostas geradas. Não precisa de respostas de referência anotadas manualmente:

MÉTRICA 1

faithfulness

A resposta está nos docs? Detecta alucinação: verifica se cada afirmação da resposta pode ser encontrada no contexto recuperado.

0 = alucinou · 1 = 100% grounded

MÉTRICA 2

answer_relevancy

Respondeu a pergunta? Mede se a resposta é relevante para o que foi perguntado — mesmo que sejá factualmente correta, pode não responder à pergunta.

0 = irrelevante · 1 = totalmente relevante

MÉTRICA 3

context_precision

Recuperou os docs certos? Avalia se os chunks recuperados são realmente relevantes para a pergunta. Mede a qualidade do retriever.

0 = ruído · 1 = todos relevantes

LLM-AS-JUDGE

O RAGAS não precisa de humanos avaliando respostas. Ele usa um LLM (o mesmo da chain ou outro) para avaliar: "Esta afirmação está fundamentada no contexto?" — gerando um score automático. É uma forma de "IA avaliando IA" — escalável e consistente.

RAGAS — implementação simplificada para o CKP02

O RAGAS oficial usa OpenAI por padrão. Para o semestre (sem API paga), implementamos as métricas principais com o Ollama local:

RAGAS_OLLAMA.PY

```
!pip install ragas -q

from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper
from datasets import Dataset

# Configurar o RAGAS para usar o Ollama (sem custo)
ragas_llm = LangchainLLMWrapper(ChatOllama(model="qwen3.6:27b", temperature=0))
ragas_embs = LangchainEmbeddingsWrapper(OllamaEmbeddings(model="nomic-embed-text"))

faithfulness.llm = ragas_llm
answer_relevancy.llm = ragas_llm
answer_relevancy.embeddings = ragas_embs

# Dataset de avaliação - 5 pares preparados antes da aula
dados_avaliação = {
    "question": [
        "Qual é o prazo de garantia do produto?",
        "Como acionar o suporte técnico?",
    ],
    "answer": [
        chain_rag.invoke("Qual é o prazo de garantia do produto?"),
        chain_rag.invoke("Como acionar o suporte técnico?"),
    ],
    "contexts": [
        [d.page_content for d in retriever.invoke("garantia")],
        [d.page_content for d in retriever.invoke("suporte técnico")],
    ],
}

resultado = evaluate(
    Dataset.from_dict(dados_avaliação),
    metrics=[faithfulness, answer_relevancy],
)

print(resultado.to_pandas()[["question", "faithfulness", "answer_relevancy"]])
```

Interpretar os scores RAGAS

O que os números significam na prática — e o que fazer quando os scores são baixos:

SCORE	FAITHFULNESS	ANSWER_RELEVANCY	DIAGNÓSTICO
0.9 – 1.0	Resposta ancorada nos docs ✓	Resposta direta à pergunta ✓	Pipeline funcionando bem
0.7 – 0.89	Pequenas extrapolações	Resposta parcialmente relevante	Ajustar prompt de grounding ou chunk_size
0.5 – 0.69	Alucinações moderadas	Resposta tangencial	Revisar prompt + verificar se chunks certos estão sendo recuperados
0.0 – 0.49	Alucina com frequência ✗	Não responde a pergunta ✗	Problema no retriever ou no prompt de grounding — rever chunking e k

⚠ FAITHFULNESS BAIXO → ALUCINAÇÃO

O modelo está inventando. Verificar: (1) o grounding está no prompt? (2) temperature=0? (3) os chunks certos estão sendo recuperados? Adicionar "Nunca invente" explicitamente no prompt.

⚡ ANSWER_RELEVANCY BAIXO → DESVIO DE FOCO

O modelo está respondendo mas não à pergunta feita. Verificar: (1) o retriever está trazendo chunks relevantes? (2) o chunk_size está capturando o contexto necessário? (3) aumentar k.

📖 FUNDAMENTAÇÃO — COMO O FAITHFULNESS É CALCULADO POR BAIXO

Polzer (*RAG with Python Cookbook*, O'Reilly, 2026) detalha o mecanismo do score de faithfulness usado pelo RAGAS: um LLM primeiro decompõe a resposta em afirmações isoladas e verificáveis — uma resposta sobre ingredientes de receita, por exemplo, vira sete afirmações do tipo "contém farinha", "contém açúcar" etc. Depois, cada afirmação é checada contra o contexto recuperado: está sustentada por ele ou não? O score final é simplesmente a proporção de afirmações sustentadas sobre o total de afirmações extraídas. Entender esse mecanismo explica por que um faithfulness baixo aponta direto para o prompt de grounding ou para o retriever: o problema está em uma dessas duas etapas, nunca na métrica em si.

Fallback — RAGAS manual se o pacote tiver problema de compatibilidade

Se o pacote `ragas` apresentar conflito de dependências no Colab, use esta implementação simplificada de faithfulness com o próprio LLM como juiz:

```
FAITHFULNESS_MANUAL.PY

# Implementação manual de faithfulness (LLM-as-judge)
PROMPT_JUIZ = """< tarefa>
Você é um avaliador de qualidade de sistemas RAG.
Avalie se a RESPOSTA está fundamentada no CONTEXTO.
</ tarefa>

< contexto>{contexto}</ contexto>
< resposta>{resposta}</ resposta>

Responda APENAS com um número de 0 a 1:
- 1.0: toda a resposta está no contexto
- 0.5: resposta parcialmente no contexto
- 0.0: resposta inventa informações não presentes no contexto

Resposta (só o número):"""

def faithfulness_manual(pergunta, resposta, contexto) -> float:
    juiz = ChatOllama(model="qwen3.6:27b", temperature=0)
    prompt = ChatPromptTemplate.from_template(PROMPT_JUIZ)
    chain_juiz = prompt | juiz | StrOutputParser()
    resultado = chain_juiz.invoke({
        "contexto": contexto, "resposta": resposta
    })
    try:
        return float(resultado.strip())
    except:
        return -1.0 # score inválido - revisar resposta do modelo

# Aplicar a todas as perguntas
scores = []
for q in PERGUNTAS:
    resp = chain_rag.invoke(q)
    ctx = "\n".join(d.page_content for d in retriever.invoke(q))
    scores.append(faithfulness_manual(q, resp, ctx))
print(f"Faithfulness médio: {sum(scores)/len(scores):.3f}")
```

RAGAS e chunking avançado em produção

AVALIAÇÃO

LLM-as-judge em escala

Provedores de modelos de linguagem usam o próprio LLM como juiz para avaliar milhões de respostas durante desenvolvimento e monitoramento contínuo. O princípio do RAGAS (IA avaliando IA) é o mesmo — só em escala de datacenter.

RETRIEVAL

Reranking como serviço gerenciado

Diversos provedores de busca oferecem reranking como API gerenciada: você envia os candidatos do retriever e recebe de volta reordenados por relevância. O mesmo princípio do cross-encoder local que você usou hoje, hospedado como serviço.

MLOPS

Monitoramento contínuo com RAGAS

Plataformas de MLOps de código aberto integram métricas do tipo RAGAS para logging automático da qualidade de RAG a cada deploy. Faithfulness e answer_relevancy viram métricas de monitoramento contínuo em produção.

Código da demo — comparar chunk_size com RAGAS

O código que o professor executa ao vivo no slide anterior — mesmo pipeline, três chunk_sizes, RAGAS medindo cada um:

DEMO_COMPARAR_CHUNKING.PY

```
def avaliar_chunking(paginas, chunk_size: int, perguntas: list) -> dict:
    """Cria pipeline RAG com chunk_size dado e retorna scores RAGAS."""
    chunks = RecursiveCharacterTextSplitter(chunk_size=chunk_size, chunk_overlap=chunk_size//8).split_documents(paginas)
    retriever = Chroma.from_documents(chunks, embeddings).as_retriever(search_kwargs={"k":3})
    chain = montar_chain_rag(retriever)

    dados = {"question": [], "answer": [], "contexts": []}
    for q in perguntas:
        dados["question"].append(q)
        dados["answer"].append(chain.invoke(q))
        dados["contexts"].append([d.page_content for d in retriever.invoke(q)])

    res = evaluate(Dataset.from_dict(dados), metrics=[faithfulness, answer_relevancy])
    return {"chunk_size": chunk_size,
            "faithfulness": res["faithfulness"],
            "answer_relevancy": res["answer_relevancy"]}

for cs in [256, 512, 1024]:
    r = avaliar_chunking(paginas, cs, PERGUNTAS_AVALIACAO)
    print(f"chunk={r['chunk_size']:4d} | faith={r['faithfulness']:.3f} | rel={r['answer_relevancy']:.3f}")
```

RESULTADO TÍPICO DA DEMO (ILUSTRATIVO)

chunk=256	faith=0.71	0.71
chunk=512 ✓	faith=0.91	0.91
chunk=1024	faith=0.83	0.83

*Valores ilustrativos — o resultado real depende do corpus do domínio

Python novo desta aula

CONCEITOS_PYTHON_AULA07_2SEM.PY

```
# 1. Divisão inteira com // – chunk_overlap como fração do chunk_size
chunk_size = 512
chunk_overlap = chunk_size // 8 # → 64 (divisão inteira, sem float)

# 2. Loop com tupla de 3 elementos – (nome, chain, retriever)
for nome, chain, retr in [
    ("A", chain_a, retriever_a),
    ("B", chain_b, retriever_b),
]:
    print(f"Testando {nome}")

# 3. try / except com ValueError para float parsing
try:
    score = float(texto.strip()) # LLM pode retornar "0.87" ou texto extra
except ValueError:
    score = -1.0 # sentinela: score inválido

# 4. List comprehension aninhado – contexts para o RAGAS
contexts = [
    [d.page_content for d in retriever.invoke(q)]
    for q in perguntas
] # lista de listas: ["chunk1", "chunk2", ...], [...], ...

# 5. sum() / len() para média simples
media = sum(scores) / len(scores) # sem precisar importar statistics

# 6. Dataset.from_dict() – criar dataset HuggingFace de um dict
from datasets import Dataset
ds = Dataset.from_dict({"col_a": [1,2], "col_b": ["x","y"]})
```

05

Lab — Notebook Aluno

55% de lacunas — otimizar o RAG do grupo com 2 estratégias de chunking, calcular RAGAS e documentar qual configuração foi melhor. Este notebook é o CKP02.

Notebook Aluno — 55% de lacunas

AULA_07_2SEM_ALUNO.IPYNB

```
!pip install langchain langchain-community langchain-ollama pymupdf chromadb ragas datasets langchain-experimental langchain-text-splitters -q

# Setup (idêntico à Aula 06)
import os
from google.colab import userdata, files
os.environ["OLLAMA_HOST"] = "https://ollama.com"
os.environ["OLLAMA_API_KEY"] = userdata.get("OLLAMA_API_KEY")

# 🚩 LACUNA 1: carregue os PDFs do grupo (mesmos da Aula 06)
paginas = []
for pdf in pdf_paths:
    paginas.extend(PyMuPDFLoader(____).load())

# — ESTRATÉGIA A: Recursive com chunk_size=512 (vs. 800 da Aula 06) —
chunks_a = RecursiveCharacterTextSplitter(
    chunk_size=____, chunk_overlap=____
).split_documents(paginas)
db_a = Chroma.from_documents(chunks_a, embeddings, persist_directory="/content/ckp02_a")
retriever_a = db_a.as_retriever(search_kwargs={"k":3})
chain_a = montar_chain_rag(retriever_a)

# — ESTRATÉGIA B: SemanticChunker —
# 🚩 LACUNA 2: instancie o SemanticChunker com o modelo de embedding
chunks_b = SemanticChunker(____).split_documents(paginas)
db_b = Chroma.from_documents(chunks_b, embeddings, persist_directory="/content/ckp02_b")
retriever_b = ____ # retriever com k=3
chain_b = montar_chain_rag(retriever_b)

# 🚩 LACUNA 3: defina 5 perguntas reais do domínio para avaliação
PERGUNTAS = [____, ____, ____, ____, ____]

# 🚩 LACUNA 4: monte o dataset RAGAS e avalie as duas estratégias
for nome, chain, retr in [
    ("Recursive-512", chain_a, retriever_a),
    ("Semantic", chain_b, retriever_b),
]:
    dados = {
        "question": PERGUNTAS,
        "answer": [chain.invoke(q) for q in PERGUNTAS],
        "contexts": [[d.page_content for d in retr.invoke(q)] for q in ____],
    }
    res = evaluate(Dataset.from_dict(dados), metrics=[faithfulness, answer_relevancy])
    print(f"{nome}: faith={res['faithfulness']:.3f} · rel={res['answer_relevancy']:.3f}")
```

🚩 CKP02 — DocMind RAG · Rubrica completa

RAG funcional sobre base de conhecimento real do domínio — 2 estratégias de chunking comparadas com RAGAS. Grupo 3–4. Estimativa: 3–4h.

🚩 CKP02 · GRUPO 3–4 · ENTREGA ESTA AULA · ESTIMATIVA: 3–4H

R1 Pipeline completo (30%): load (PyMuPDF) → split → embed (nomic) → store (ChromaDB) → retrieve → generate (qwen3.6:27b).
Responde com citação de página e fonte. *Baseline da Aula 06.*

R2 2 estratégias de chunking (30%): Recursive em pelo menos 2 chunk_sizes diferentes (ex: 512 e 1024) OU Recursive vs. SemanticChunker. Cada estratégia deve ter seu próprio vector store e retriever.

R3 RAGAS documentado (25%): faithfulness e answer_relevancy medidos para as 2 estratégias com mínimo de 5 perguntas. Célula markdown explicando qual estratégia ganhou e por que.

R4 Análise crítica (15%): identificar 1 pergunta onde o pipeline falhou (score baixo). Mostrar os chunks recuperados e hipótese do grupo sobre a causa da falha. Propor uma melhoria.

📦 ENTREGÁVEL

1 notebook .ipynb com os 4 requisitos.
Nome: CKP02_[dominio]_grupo.ipynb
Link Colab com permissão de leitura.

★ DIFERENCIAL

Metadata filtering com where={} + reranking com cross-encoder + interface Gradio com campo de pergunta e exibição dos chunks recuperados.

Próximo: Módulo 3 — Interfaces, Agentes e Integração



Módulo 2 concluído: Embeddings · ChromaDB · Pipeline RAG · Chunking · Reranking · RAGAS · CKP02 entregue

Módulo 3 — Interfaces · Agentes · Integração (Aulas 08–11)

- A8** Gradio + Streamlit — RAG com URL pública. RunnableWithMessageHistory para memória.
- A9** Agentes ReAct — tools, function calling, planner-executor.
- A10** RAG como tool — agente que usa o ChromaDB do grupo como ferramenta.
- A11** Aula integradora — agente completo + Gradio URL pública. **CKP03.**



PREPARE-SE PARA A AULA 08

A Aula 08 coloca o RAG ao vivo com Gradio. O notebook do CKP02 (funcionando no Colab) é o ponto de partida. Salve o link do Colab e mantenha as permissões de acesso abertas.

Check-in intermediário — metade do semestre

Aula 07 marca a metade do semestre (Módulos 1 e 2 concluídos). O professor faz 2 perguntas anônimas rápidas — resultados lidos ao vivo na Aula 08:



Pergunta 1

Qual conceito do semestre até agora foi mais difícil de entender?

(resposta anônima via QR Code ou link)



Pergunta 2

O que você mais quer aprender nos Módulos 3 e 4 (agentes, LangGraph, tendências)?

(resposta anônima via QR Code ou link)

POR QUE O CHECK-IN IMPORTA

O feedback anônimo permite ajustar a velocidade e profundidade dos Módulos 3 e 4. Se a maioria teve dificuldade com LCEL (Módulo 1), a Aula 08 reforça antes de avançar. Se todos querem ver mais agentes, o Módulo 3 ganha mais tempo.

04

Demo ao vivo

Mesmo RAG, mesmo corpus, mesmo prompt — três configurações de chunk_size: 256, 512 e 1024. Medir RAGAS e mostrar qual ganhou.



Demo Prática ao Vivo

O professor roda o mesmo pipeline RAG sobre o mesmo corpus e as mesmas perguntas, variando apenas o `chunk_size` — 256, 512 e 1024 — e mede `faithfulness` e `answer_relevancy` com RAGAS ao vivo para revelar qual configuração realmente venceu.

1. TRÊS PIPELINES, UM CORPUS

O mesmo conjunto de PDFs é indexado três vezes — uma para cada `chunk_size` — gerando três vector stores independentes no ChromaDB.

2. RAGAS MEDE CADA UM

As mesmas perguntas de avaliação rodam contra os três pipelines. RAGAS calcula `faithfulness` e `answer_relevancy` para cada `chunk_size`.

3. O PLACAR DECIDE

Os três scores aparecem lado a lado — a turma vê ao vivo qual `chunk_size` teve o melhor equilíbrio entre contexto e precisão para este corpus.

🛠️ STACK DA DEMO

Google Colab · Ollama Cloud API · ChromaDB · RAGAS · qwen3.6:27b

Otimizar RAG com RAGAS — 2 estratégias de chunking ★★★

Grupo 3–4 · 20 minutos · Google Colab · PDFs do domínio obrigatórios

1. **Complete as 4 lacunas** — carregar PDFs, instanciar SemanticChunker, montar retriever_b e preencher PERGUNTAS com 5 perguntas reais do domínio.
2. **Compare as duas estratégias** — documente em uma célula markdown qual obteve melhores scores de faithfulness e answer_relevancy e por que (hipótese do grupo).
3. **Análise de falhas:** identifique 1 pergunta onde a chain errou (faithfulness baixo) — recupere os chunks que foram usados e explique por que o retriever não trouxe o trecho certo.

🎯 Gabarito das lacunas

Lacuna 1: `PyMuPDFLoader(pdf).load()`

Lacuna 2: `SemanticChunker(embeddings)`

Lacuna 3: `db_b.as_retriever(search_kwargs={"k": 3})`

Lacuna 4: `PERGUNTAS` (no loop de contexts)

★ **Diferencial CKP02:** adicionar metadata filtering por categoria de documento + reranking com cross-encoder. Implementar uma interface Gradio simples para o RAG (slide de referência na Aula 08).

Referências

PAPER

Es, S. et al. — "RAGAS: Automated Evaluation of Retrieval Augmented Generation." EACL, 2024. O paper que define faithfulness e answer_relevancy. arxiv.org/abs/2309.15217

DOCS

RAGAS — Documentação oficial: métricas, integração com Ollama, datasets. docs.ragas.io

DOCS

LangChain — SemanticChunker e ParentDocumentRetriever. python.langchain.com/docs/how_to/semantic-chunker

MODELO

cross-encoder/ms-marco-MiniLM-L-6-v2 — Modelo de reranking leve (22M params). huggingface.co/cross-encoder/ms-marco-MiniLM-L-6-v2

LIVRO

Goodfellow, I.; Bengio, Y.; Courville, A. — Deep Learning. Pearson, 2017. Cap. 15 — Representações distribuídas: a base teórica dos embeddings que sustentam o chunking semântico.

Módulo 2 concluído

RAG com qualidade medida e documentada. Agora é hora de colocar ao vivo — Gradio, URL pública, e transformar em agente autônomo.



Chunking é uma variável de qualidade. `chunk_size=800` é um ponto de partida — o valor ótimo depende do domínio, da densidade do documento e do modelo de embedding.



Três estratégias de chunking — Recursive (padrão do semestre), Semantic (coerência por embedding) e Parent-Document Retriever (busca precisa, contexto rico). Cada uma para um cenário diferente.



Reranking com cross-encoder melhora a precisão sem mudar a chain — só substitui o retriever. Recupera 10, reordena, envia os 3 melhores.



RAGAS transforma "parece bom" em "é bom com score 0.91". `faithfulness` mede alucinação. `answer_relevancy` mede foco. LLM-as-judge escala sem anotação humana.



CKP02 entregue — Módulo 2 concluído. O grupo tem um sistema RAG sobre documentos reais do domínio, com qualidade medida e documentada. Módulo 3: colocar isso ao vivo com Gradio e transformar em agente.

→ [PRÓXIMA AULA](#) — [AULA 08](#) · 29/SET

Interfaces com Gradio e Streamlit

RAG com URL pública. RunnableWithMessageHistory para memória entre turnos.

[Trazer para a Aula 08](#)

Notebook CKP02 funcionando. Link do Colab com acesso aberto. A Aula 08 pega o RAG daqui e coloca ao vivo.